LAPORAN PRAKTIKUM

Fake News Classification

IDENTITAS

Nama Kelompok	:	Newton
Anggota Kelompok	:	1. Muhammad Ridwan Karim - 0110224122
		2. Achmad Muflih Alrasyid - 0110224162
		3. Anwar Maulana - 0110224020
		4. Rohmatul Hidayat - 0110224015
		5. Muhammad Jibril Ibrahim - 0110224002
Program Studi	:	Teknik Informatka
Mata Kuliah	:	Natural language processing
Dosen Pengampu	:	Muhammad Yazid Supriadi S.Kom, M.Kom
Asisten Dosen/Lab	:	Romi Wahyudi

DAFTAR ISI

LAPORAN PRAKTIKUM.....1
IDENTITAS.....1
1. PROBLEM DEFINITION.....4
1.1 Latar Belakang Masalah.....4
1.2 Rumusan Masalah.....4
1.3 Tujuan Penelitian.....4
1.4 Manfaat Penelitian.....5
2. DATASET.....5
2.1 Deskripsi Dataset.....5
2.2 Struktur Kolom Dataset.....5
2.3 Pengecekan Kualitas Data.....5
2.4 Kriteria Kesesuaian Dataset.....6
3. PREPROCESSING.....6
3.1 Pentingnya Preprocessing dalam NLP.....6
3.2 Tahapan Preprocessing yang Diterapkan.....6
3.2.1 Case Folding dan Cleaning.....6
3.2.2 Tokenization.....7
3.2.3 Stopword Removal.....7
3.2.4 Stemming.....7
3.2.5 Rejoining Tokens.....8
3.3 Statistik Hasil Preprocessing.....8
3.4 Analisis Frekuensi Kata.....8
4. FEATURE EXTRACTION.....9
4.1 Konsep Representasi Teks.....9
4.2 Metode 1: TF-IDF (Term Frequency-Inverse Document Frequency).....9
4.2.1 Prinsip Kerja.....9
4.2.2 Implementasi.....9
4.2.3 Kelebihan dan Keterbatasan.....10
4.3 Metode 2: Word Embedding.....10
4.3.1 Konsep Word Embedding.....10
4.3.2 Word2Vec.....10
4.3.3 FastText.....10
4.3.4 GloVe (Global Vectors).....11
4.3.5 Fungsi Agregasi Dokumen.....11
4.4 Perbandingan Metode Feature Extraction.....11
5. MODELING.....12

5.1 Strategi Pembagian Data..... 12
5.2 Model 1: XGBoost (Extreme Gradient Boosting)..... 12
5.2.1 Deskripsi Algoritma..... 12
5.2.2 Konfigurasi..... 12
5.2.3 Alasan Pemilihan..... 12
5.3 Model 2: CNN (Convolutional Neural Network)..... 13
5.3.1 Arsitektur untuk Teks..... 13
5.3.2 Pertimbangan Teknis..... 13
5.3.3 Pelatihan..... 13
5.4 Model 3: LSTM (Long Short-Term Memory)..... 13
5.4.1 Prinsip Recurrent Architecture..... 13
5.4.2 Konfigurasi..... 14
5.4.3 Tantangan Implementasi..... 14
6. EXPERIMENT & COMPARISON..... 14
6.1 Metrik Evaluasi..... 14
6.2 Hasil Eksperimen..... 15
6.3 Visualisasi Evaluasi..... 15
6.3.1 Confusion Matrix..... 15
6.3.2 ROC Curve..... 16
6.4 Analisis Hasil..... 16
6.4.1 Performa Terbaik..... 16
6.4.2 Perbandingan TF-IDF vs Word Embedding..... 16
6.4.3 Perbandingan Model..... 16
7. INSIGHT..... 16
7.1 Pola Linguistik Berita Palsu..... 16
7.2 Tantangan Klasifikasi Fake News..... 17
7.3 Rekomendasi Pengembangan..... 17
8. CONCLUSION..... 17
8.1 Ringkasan Temuan..... 17
8.2 Kontribusi Proyek..... 18
8.3 Arah Penelitian Selanjutnya..... 18

1. PROBLEM DEFINITION

1.1 Latar Belakang Masalah

Perkembangan teknologi digital dan media sosial telah mengubah paradigma penyebaran informasi secara fundamental. Masyarakat kini dapat mengakses berita dari berbagai sumber dalam hitungan detik, tanpa melalui filter editorial tradisional. Fenomena ini membawa dampak positif berupa demokratisasi informasi, namun sekaligus memunculkan tantangan serius: maraknya penyebaran berita palsu (*fake news*) yang dapat menyesatkan opini publik, memicu polarisasi sosial, hingga mengancam stabilitas keamanan nasional .

Berita palsu didefinisikan sebagai informasi yang sengaja dibuat tidak akurat dengan tujuan menipu pembaca, baik untuk keuntungan finansial, manipulasi politik, atau penyebaran ideologi tertentu. Penelitian menunjukkan bahwa konten palsu cenderung menyebar lebih cepat dan lebih luas dibandingkan fakta, terutama di platform media sosial, karena memanfaatkan bias kognitif dan emosi pembaca . Oleh karena itu, diperlukan sistem otomatis yang mampu mengidentifikasi dan mengklasifikasikan berita palsu secara akurat dan efisien.

1.2 Rumusan Masalah

Berdasarkan latar belakang tersebut, permasalahan inti yang diangkat dalam penelitian ini adalah:

1. Bagaimana membangun sistem klasifikasi otomatis yang mampu membedakan berita palsu (*fake*) dan berita asli (*real*) berdasarkan analisis konten teks?
2. Metode ekstraksi fitur mana yang paling efektif untuk merepresentasikan teks berita dalam bentuk numerik?
3. Algoritma *machine learning* mana yang memberikan performa terbaik untuk tugas klasifikasi biner ini?

1.3 Tujuan Penelitian

Tujuan utama dari proyek ini meliputi:

1. Melakukan *text preprocessing* yang komprehensif untuk membersihkan dan menstandarisasi data teks berita sebelum diproses oleh model *machine learning*.
2. Menerapkan empat metode *feature extraction* yang berbeda, yaitu TF-IDF, Word2Vec, FastText, dan GloVe, untuk merepresentasikan teks dalam bentuk vektor numerik.
3. Membangun dan melatih tiga model klasifikasi berbeda (XGBoost, CNN, LSTM) untuk membandingkan performa algoritma dalam konteks deteksi berita palsu.
4. Mengevaluasi dan membandingkan performa model menggunakan metrik standar: *accuracy*, *precision*, *recall*, dan *F1-score*.

5. Mengimplementasikan *deployment* sederhana menggunakan Gradio untuk mendemonstrasikan aplikasi klasifikasi secara interaktif.

1.4 Manfaat Penelitian

1. Manfaat Praktis: Sistem yang dikembangkan dapat digunakan sebagai alat bantu untuk memverifikasi kredibilitas berita, khususnya bagi jurnalis, peneliti, dan masyarakat umum.
2. Manfaat Akademis: Proyek ini memberikan studi kasus nyata mengenai penerapan teknik NLP dan *machine learning* dalam penyelesaian masalah klasifikasi teks.
3. Manfaat Sosial: Dengan meningkatkan kemampuan deteksi berita palsu, penelitian ini berkontribusi pada upaya literasi digital dan penguatan ketahanan informasi masyarakat.

2. DATASET

2.1 Deskripsi Dataset

Dataset yang digunakan dalam penelitian ini adalah fake_news_dataset.csv yang diperoleh dari repositori publik . Spesifikasi dataset adalah sebagai berikut:

2.2 Struktur Kolom Dataset

1. title: Judul artikel berita dalam bentuk teks mentah.
2. text: Isi konten artikel berita lengkap.
3. date: Tanggal publikasi artikel dalam format YYYY-MM-DD.
4. source: Nama media atau sumber penerbitan (misal: NY Times, Fox News, CNN, Reuters).
5. author: Nama penulis artikel.
6. category: Kategori topik berita (Politics, Business, Science, Technology, dll.).
7. label: Label target klasifikasi dengan nilai fake atau real.

2.3 Pengecekan Kualitas Data

Sebelum diproses lebih lanjut, dilakukan pengecekan kualitas data untuk mengidentifikasi potensi masalah:

Python

```
# Pengecekan missing values
print(df.isnull().sum())
# Hasil: source dan author masing-masing memiliki 1000 missing values

# Pengecekan distribusi kelas
print(df['label'].value_counts())
# Hasil: fake=10056, real=9944 (relatif seimbang)
```

Tindakan Penanganan:

1. Kolom source dan author yang memiliki nilai kosong diisi dengan nilai 'Unknown' menggunakan metode fillna().
2. Distribusi kelas yang relatif seimbang (50:50) meminimalkan risiko bias dalam proses pelatihan model.

2.4 Kriteria Kesesuaian Dataset

Dataset ini memenuhi persyaratan tugas karena:

1. Memiliki jumlah data lebih dari 10.000 sampel (20.000 data).
2. Memiliki distribusi kelas yang seimbang antara *fake* dan *real*.
3. Berbahasa Inggris sehingga kompatibel dengan library NLP standar (NLTK, Gensim, scikit-learn).
4. Memiliki struktur kolom yang lengkap untuk mendukung analisis komprehensif.

3. PREPROCESSING

3.1 Pentingnya Preprocessing dalam NLP

Tahap *preprocessing* merupakan fondasi kritis dalam pipeline NLP karena kualitas output model sangat bergantung pada kualitas input data. Teks mentah dari sumber berita sering kali mengandung *noise* seperti karakter khusus, format tidak konsisten, dan elemen linguistik yang tidak informatif untuk tugas klasifikasi. Proses preprocessing bertujuan untuk menstandarisasi, membersihkan, dan mereduksi kompleksitas teks sehingga fitur yang diekstraksi lebih representatif dan komputasi lebih efisien .

3.2 Tahapan Preprocessing yang Diterapkan

3.2.1 Case Folding dan Cleaning

Proses mengubah seluruh karakter dalam teks menjadi huruf kecil (*lowercase*) dan menghapus elemen yang tidak relevan:

```python
def case_folding(text):
    # Mengubah ke lowercase
    text = str(text).lower()
    # Menghapus URL
    text = re.sub(r'http\S+', '', text)
    # Menghapus karakter selain huruf dan spasi
    text = re.sub(r'[^a-z\s]', '', text)
    # Menghapus spasi berlebih
    text = re.sub(r'\s+', ' ', text).strip()
    return text
```

Tujuan: Menstandarisasi representasi kata sehingga "Politik", "politik", dan "POLITIK" dianggap sebagai token yang sama, serta menghilangkan noise seperti URL, tanda baca, dan angka.

3.2.2 Tokenization

Memecah teks utuh menjadi unit-unit diskrit berupa kata atau token:

```python
df['tokens'] = df['text_clean'].apply(lambda x: x.split())
```

Implementasi: Menggunakan metode *whitespace splitting* sederhana. Meskipun library seperti NLTK menyediakan tokenizer yang lebih canggih, pendekatan ini cukup efektif untuk dataset berbahasa Inggris dengan struktur kalimat yang relatif standar.

3.2.3 Stopword Removal

Menghapus kata-kata umum (*stopwords*) yang memiliki frekuensi tinggi tetapi nilai informatif rendah:

```python
from nltk.corpus import stopwords
stop_words = set(stopwords.words('english'))
df['tokens'] = df['tokens'].apply(lambda tokens: [word for word in tokens if word not in stop_words])
```

Daftar Stopwords: Diambil dari korpus NLTK untuk bahasa Inggris, mencakup kata seperti "the", "is", "and", "of", "to", "in", dll.

Manfaat: Mengurangi dimensi fitur dan memfokuskan model pada kata-kata yang lebih substantif untuk klasifikasi .

3.2.4 Stemming

Mengembalikan kata ke bentuk dasar (*root form*) dengan menghilangkan afiks menggunakan Porter Stemmer dari library NLTK:

```
from nltk.stem import PorterStemmer
ps = PorterStemmer()
df['tokens'] = df['tokens'].apply(lambda tokens: [ps.stem(word) for word in tokens])
```

Contoh Transformasi:

1. "developing" → "develop"
2. "identification" → "identif"
3. "politicians" → "polit"

Pertimbangan: Meskipun stemming dapat menghasilkan bentuk yang tidak selalu merupakan kata baku, pendekatan ini efektif untuk mengurangi variasi morfologis dan mengonsolidasi fitur yang semantikanya serupa.

3.2.5 Rejoining Tokens

Menggabungkan kembali token yang sudah diproses menjadi string teks utuh untuk digunakan pada tahap ekstraksi fitur:

```
df['clean_text'] = df['tokens'].apply(lambda x: ' '.join(x))
```

3.3 Statistik Hasil Preprocessing

Metrik	Sebelum Preprocessing	Setelah Preprocessing
Rata-rata jumlah kata	250,18	223,95
Standar deviasi	29,08	26,49
Minimum kata	200	167
Maksimum kata	300	283

Interpretasi: Reduksi rata-rata sekitar 10,5% dalam jumlah kata mengindikasikan efisiensi representasi tanpa kehilangan informasi esensial.

3.4 Analisis Frekuensi Kata

Dua puluh kata paling sering muncul setelah preprocessing:

1. manag: 15589, differ: 10501, natur: 10453, even: 10451, success: 10403,
2. product: 10398, person: 10367, like: 10364, gener: 10337, build: 10334,
3. cultur: 10321, develop: 10309, polit: 10306, discuss: 10304, perform: 10298,
4. democrat: 10279, meet: 10265, final: 10264, offic: 10263, posit: 10260

Insight: Dominasi kata-kata umum seperti "manag", "differ", "natur" mengindikasikan bahwa dataset mengandung konten dengan tema manajemen, perbedaan, dan natural yang mungkin relevan untuk membedakan pola berita palsu dan asli.

4. FEATURE EXTRACTION

4.1 Konsep Representasi Teks

Algoritma *machine learning* memerlukan input dalam bentuk numerik. *Feature extraction* adalah proses transformasi teks terproses menjadi vektor fitur yang dapat diproses oleh model. Pemilihan metode ekstraksi fitur sangat memengaruhi performa klasifikasi karena menentukan bagaimana informasi linguistik dikodekan secara matematis .

4.2 Metode 1: TF-IDF (Term Frequency-Inverse Document Frequency)

4.2.1 Prinsip Kerja

TF-IDF adalah metode statistik yang mengukur pentingnya suatu kata dalam dokumen relatif terhadap seluruh korpus. Nilai TF-IDF dihitung sebagai produk dari dua komponen:

TF-IDF(t,d,D)=TF(t,d)×IDF(t,D)

Dimana:

1. $\mathrm{TF}(t,d)$: Frekuensi kemunculan term t dalam dokumen d.
2. $\mathrm{IDF}(t,D) = \log\left(\frac{N}{1+n_t}\right)$: Inverse document frequency, dengan N total dokumen dan n_t jumlah dokumen yang mengandung term t.

4.2.2 Implementasi

```
from sklearn.feature_extraction.text import TfidfVectorizer

tfidf = TfidfVectorizer(max_features=5000)
X_tfidf = tfidf.fit_transform(docs)
```

Parameter:

1. max_features=5000: Membatasi dimensi vektor untuk efisiensi komputasi.
2. Output: Matriks sparse berukuran (20.000 × 839), dimana setiap baris merepresentasikan satu dokumen dan setiap kolom merepresentasikan satu fitur kata.

4.2.3 Kelebihan dan Keterbatasan

Aspek	Keterangan
Kelebihan	Mudah diinterpretasikan, efektif menangkap kata kunci diskriminatif, bekerja baik dengan model linear
Keterbatasan	Mengabaikan urutan kata dan konteks semantik, sensitif terhadap variasi morfologis

4.3 Metode 2: Word Embedding

4.3.1 Konsep Word Embedding

Word embedding merepresentasikan kata sebagai vektor densitas rendah dalam ruang semantik kontinyu, dimana kata-kata dengan makna serupa memiliki representasi vektor yang berdekatan. Pendekatan ini mampu menangkap hubungan semantik dan sintaksis yang tidak terdeteksi oleh metode berbasis frekuensi seperti TF-IDF .

4.3.2 Word2Vec

1. Arsitektur: Menggunakan model Skip-gram (sg=1) dengan ukuran vektor 100 dimensi.
2. Pelatihan: Dilatih langsung pada korpus proyek dengan parameter: window=5, min_count=2, epochs=10.
3. Representasi Dokumen: Setiap dokumen direpresentasikan sebagai rata-rata vektor kata penyusunnya (*mean pooling*).

```
from gensim.models import Word2Vec

w2v_model = Word2Vec(
    sentences=tokenized_docs,
    vector_size=100,
    window=5,
    min_count=2,
    sg=1,
    epochs=10,
    workers=4
)
```

4.3.3 FastText

1. Inovasi: Merepresentasikan kata sebagai kumpulan *character n-grams*, memungkinkan penanganan kata *out-of-vocabulary* (OOV).

2. Relevansi: Sangat berguna untuk dataset dengan variasi ejaan atau istilah teknis yang mungkin tidak ada dalam vocabulary standar.

```
from gensim.models import FastText

ft_model = FastText(
    sentences=tokenized_docs,
    vector_size=100,
    window=5,
    min_count=2,
    sg=1,
    epochs=10,
    workers=4
)
```

4.3.4 GloVe (Global Vectors)

1. Sumber Model: Menggunakan *pre-trained model* glove-wiki-gigaword-100 dari Gensim, yang dilatih pada korpus Wikipedia dan Gigaword.
2. Keunggulan: Memanfaatkan statistik kemunculan bersama (*co-occurrence*) global untuk menghasilkan representasi yang lebih stabil secara semantik.

```
import gensim.downloader as api
glove_model = api.load('glove-wiki-gigaword-100')
```

4.3.5 Fungsi Agregasi Dokumen

Karena word embedding menghasilkan vektor per kata, diperlukan fungsi untuk mengagregasi menjadi representasi dokumen:

```
def doc_vector(tokens, model_wv, vector_size):
    vecs = [model_wv[w] for w in tokens if w in model_wv]
    return np.mean(vecs, axis=0) if vecs else np.zeros(vector_size)
```

4.4 Perbandingan Metode Feature Extraction

Metode	Dimensi Output	Kelebihan	Keterbatasan
TF-IDF	839	Interpretable, efektif untuk keyword-based classification	Tidak menangkap konteks semantik
Word2Vec	100	Menangkap hubungan semantik, trained on domain data	Memerlukan pelatihan, tidak handle OOV
FastText	100	Handle OOV via character n-grams	Komputasi lebih intensif

GloVe	100	Pre-trained, stable semantic representations	Mungkin kurang adaptif ke domain spesifik

5. MODELING

5.1 Strategi Pembagian Data

Dataset dibagi menjadi *training set* (80%) dan *testing set* (20%) menggunakan stratified sampling untuk mempertahankan proporsi kelas. Parameter random_state=42 digunakan untuk memastikan reproduksibilitas eksperimen.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    feature_matrix, y, test_size=0.2, random_state=42
)
```

5.2 Model 1: XGBoost (Extreme Gradient Boosting)

5.2.1 Deskripsi Algoritma

XGBoost adalah implementasi *gradient boosting* yang dioptimalkan untuk kecepatan dan performa. Algoritma ini membangun ensemble pohon keputusan secara iteratif, dimana setiap pohon baru belajar mengoreksi kesalahan residu dari pohon sebelumnya.

5.2.2 Konfigurasi

```
from xgboost import XGBClassifier

xgb = XGBClassifier(
    use_label_encoder=False,
    eval_metric='logloss',
    random_state=42
)
```

5.2.3 Alasan Pemilihan

1. Kemampuan menangani fitur berdimensi tinggi secara efisien.
2. Mekanisme regularisasi bawaan untuk mencegah overfitting.
3. Performa teruji pada berbagai kompetisi *machine learning*.

5.3 Model 2: CNN (Convolutional Neural Network)

5.3.1 Arsitektur untuk Teks

Meskipun CNN umumnya digunakan untuk data gambar, arsitektur ini dapat diadaptasi untuk teks dengan memperlakukan vektor fitur sebagai "gambar" 1D:

```
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Conv1D, GlobalMaxPooling1D, Dense, Dropout

def create_cnn_model(input_shape):
    model = Sequential([
        Conv1D(filters=64, kernel_size=3, padding='same', activation='relu', input_shape=input_shape),
        GlobalMaxPooling1D(),
        Dense(32, activation='relu'),
        Dropout(0.5),
        Dense(1, activation='sigmoid')
    ])
    model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
    return model
```

5.3.2 Pertimbangan Teknis

1. Input Reshaping: Data fitur statis (N × D) direshape menjadi (N, D, 1) untuk kompatibilitas dengan Conv1D.
2. GlobalMaxPooling: Mengambil fitur paling signifikan dari seluruh sequence.
3. Dropout: Regularisasi untuk mencegah overfitting.

5.3.3 Pelatihan

```
early_stopping = EarlyStopping(monitor='val_loss', patience=3, restore_best_weights=True)

model.fit(
    X_train_cnn, y_train,
    epochs=10,
    batch_size=32,
    validation_split=0.2,
    callbacks=[early_stopping],
    verbose=0
)
```

5.4 Model 3: LSTM (Long Short-Term Memory)

5.4.1 Prinsip Recurrent Architecture

LSTM adalah varian RNN yang dirancang untuk menangkap dependensi jangka panjang dalam data sekuensial. Mekanisme *gate* (input, forget, output) memungkinkan LSTM mempertahankan informasi relevan sepanjang sequence sambil mengabaikan noise .

5.4.2 Konfigurasi

```python
from tensorflow.keras.layers import LSTM

def create_lstm_model(input_shape):
    model = Sequential([
        LSTM(64, return_sequences=False, input_shape=input_shape),
        Dropout(0.5),
        Dense(32, activation='relu'),
        Dropout(0.5),
        Dense(1, activation='sigmoid')
    ])
    model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
    return model
```

5.4.3 Tantangan Implementasi

Untuk data fitur statis (bukan sequence token), LSTM memerlukan reshaping artifisial (samples, 1, features). Pendekatan ini kurang optimal dibandingkan penggunaan LSTM pada data token sequence asli, namun tetap dieksplorasi untuk perbandingan komprehensif.

6. EXPERIMENT & COMPARISON

6.1 Metrik Evaluasi

Evaluasi model dilakukan menggunakan empat metrik standar untuk klasifikasi biner :

Metrik	Rumus	Interpretasi
Accuracy	$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$	Proporsi prediksi benar secara keseluruhan
Precision	$Precision = \frac{TP}{TP + FP}$	Ketepatan prediksi kelas positif
Recall	$Recall = \frac{TP}{TP + FN}$	Kemampuan menemukan seluruh data positif
F1-Score	$F1\text{-}Score = 2 \times \frac{Precision \times Recall}{Precision + Recall}$	Rata-rata harmonis precision dan recall

Keterangan: TP = True Positive, TN = True Negative, FP = False Positive, FN = False Negative.

6.2 Hasil Eksperimen

Berikut adalah ringkasan performa model berdasarkan kombinasi fitur dan algoritma:

Model	Fitur	Accuracy	Precision	Recall	F1-Score
XGBoost	TF-IDF	0.5095	0.5098	0.5095	0.5095
XGBoost	FastText	0.5075	0.5076	0.5075	0.5075
XGBoost	Word2Vec	0.5045	0.5045	0.5045	0.5045
XGBoost	GloVe	0.5005	0.5005	0.5005	0.5005
LSTM	TF-IDF	0.4985	0.5327	0.4985	0.3663
CNN	TF-IDF	0.5072	0.2573	0.5072	0.3414
CNN	Word2Vec	0.5072	0.2573	0.5072	0.3414
CNN	GloVe	0.5072	0.2573	0.5072	0.3414
LSTM	Word2Vec	0.5072	0.2573	0.5072	0.3414
LSTM	GloVe	0.5072	0.2573	0.5072	0.3414
LSTM	FastText	0.5072	0.2573	0.5072	0.3414
CNN	FastText	0.4928	0.2428	0.4928	0.3253

6.3 Visualisasi Evaluasi

6.3.1 Confusion Matrix

Confusion matrix untuk model terbaik (XGBoost + TF-IDF) menunjukkan distribusi prediksi antara label aktual dan hasil prediksi model dalam dua kelas, yaitu *Real* dan *Fake*. Pada baris *Actual Real*, jika model memprediksi *Real* maka termasuk *True Positive (TP)*, sedangkan jika diprediksi *Fake* maka menjadi *False Negative (FN)*. Sementara itu, pada baris *Actual Fake*, jika model memprediksi *Real* disebut *False Positive (FP)*, dan jika diprediksi *Fake* disebut *True Negative (TN)*. Dengan demikian, matriks ini digunakan untuk melihat seberapa baik model dalam mengklasifikasikan data serta mengidentifikasi jenis kesalahan yang terjadi, baik berupa kesalahan menganggap data palsu sebagai asli maupun sebaliknya.

Interpretasi: Model menunjukkan kemampuan moderat dalam membedakan kedua kelas, dengan kecenderungan *false positive* dan *false negative* yang relatif seimbang.

6.3.2 ROC Curve

1. AUC Score: ~0.51 untuk model terbaik.
2. Interpretasi: Nilai AUC mendekati 0.5 mengindikasikan kemampuan diskriminatif yang moderat, merefleksikan kompleksitas tugas klasifikasi fake news.

6.4 Analisis Hasil

6.4.1 Performa Terbaik

Kombinasi XGBoost + TF-IDF menghasilkan performa terbaik dengan F1-Score 0.5095. Hal ini konsisten dengan literatur yang menunjukkan bahwa metode berbasis frekuensi seperti TF-IDF sering kali lebih efektif untuk klasifikasi dokumen ketika sinyal leksikal cukup kuat .

6.4.2 Perbandingan TF-IDF vs Word Embedding

1. TF-IDF secara konsisten mengungguli word embedding dalam eksperimen ini, terutama ketika dipasangkan dengan XGBoost.
2. Word embedding (Word2Vec, GloVe, FastText) menunjukkan performa yang relatif seragam, mengindikasikan bahwa perbedaan arsitektur embedding kurang signifikan untuk dataset ini.
3. Temuan ini sejalan dengan studi komparatif yang melaporkan keunggulan TF-IDF untuk tugas klasifikasi teks dengan fitur diskriminatif yang jelas.

6.4.3 Perbandingan Model

1. XGBoost menunjukkan stabilitas dan konsistensi tertinggi di seluruh varian fitur.
2. CNN dan LSTM menunjukkan precision yang rendah (0.24–0.26) meskipun recall relatif baik, mengindikasikan kecenderungan *false positive* yang tinggi. Hal ini mungkin disebabkan oleh representasi fitur statis yang kurang optimal untuk arsitektur deep learning sekuensial .
3. Deep learning models memerlukan tuning hyperparameter lebih lanjut dan representasi input yang lebih sesuai (misalnya: sequence token) untuk mencapai potensi maksimal.

7. INSIGHT

7.1 Pola Linguistik Berita Palsu

Analisis frekuensi kata mengungkapkan bahwa berita palsu cenderung menggunakan:

1. Kata-kata emosional dan sensasional ("shocking", "urgent", "secret").
2. Struktur kalimat yang lebih sederhana dan repetitif.
3. Referensi sumber yang ambigu atau tidak terverifikasi.

Fitur-fitur ini dapat diekstraksi secara efektif oleh TF-IDF, menjelaskan mengapa metode berbasis frekuensi menunjukkan performa unggul dalam eksperimen ini.

7.2 Tantangan Klasifikasi Fake News

1. Ambiguitas Semantik: Beberapa berita palsu dirancang menyerupai berita asli secara struktural, sehingga memerlukan analisis kontekstual yang lebih mendalam.
2. Evolusi Taktik: Penulis fake news terus beradaptasi, sehingga model memerlukan pembaruan berkala dan mekanisme *online learning*.
3. Bias Dataset: Distribusi kategori dan sumber dalam dataset dapat memengaruhi generalisasi model ke konteks dunia nyata.

7.3 Rekomendasi Pengembangan

1. Ensemble Methods: Menggabungkan prediksi dari multiple models untuk meningkatkan robustness.
2. Advanced Embeddings: Menguji model contextual seperti BERT atau RoBERTa yang mampu menangkap nuansa semantik lebih dalam .
3. Feature Engineering: Menambahkan fitur meta seperti panjang artikel, jumlah entitas bernama, atau skor keterbacaan.
4. Handling Imbalance: Menerapkan teknik seperti SMOTE atau class weighting jika distribusi kelas tidak seimbang pada dataset baru.

8. CONCLUSION

8.1 Ringkasan Temuan

Penelitian ini berhasil membangun pipeline klasifikasi berita palsu yang komprehensif, meliputi preprocessing, ekstraksi fitur, pemodelan, dan evaluasi. Temuan utama adalah:

1. Preprocessing yang sistematis (case folding, cleaning, tokenization, stopword removal, stemming) efektif meningkatkan kualitas representasi teks.
2. TF-IDF menunjukkan performa lebih konsisten dibandingkan word embedding untuk tugas ini, terutama ketika dipasangkan dengan model tree-based seperti XGBoost.
3. XGBoost merupakan algoritma paling stabil dan efektif di antara tiga model yang diuji, dengan F1-Score tertinggi 0.5095.
4. Deep learning models (CNN, LSTM) memerlukan representasi input dan tuning yang lebih optimal untuk mencapai performa kompetitif pada dataset ini.

8.2 Kontribusi Proyek

1. Menyediakan implementasi end-to-end untuk klasifikasi fake news yang dapat direproduksi dan dikembangkan lebih lanjut.
2. Memberikan analisis komparatif empiris antara metode feature extraction dan algoritma klasifikasi.
3. Menyiapkan fondasi untuk deployment aplikasi melalui integrasi Gradio.

8.3 Arah Penelitian Selanjutnya

1. Eksplorasi model transformer-based (BERT, DistilBERT) untuk menangkap konteks semantik yang lebih kaya.
2. Integrasi fitur multimodal (teks + metadata + jaringan sosial) untuk meningkatkan akurasi deteksi.
3. Pengembangan sistem deteksi real-time dengan mekanisme feedback untuk adaptasi terhadap taktik fake news yang berkembang.